WEEK 2 · LESSON 8 OF 24 · HANDOUT

Pandas: Tables of Data

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=ZyhVh-qRZPA — open the lesson online to watch inside the course.

Your goal in this lesson: Load data into a Pandas DataFrame, inspect it, select columns and rows, filter, sort and create new columns.

A DataFrame is a table

Pandas is like Excel inside Python. Its main object, the **DataFrame**, is a table with named columns. Each column is a **Series**.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Aline", "Eric", "Grace", "Jean"],
    "district": ["Gasabo", "Huye", "Gasabo", "Musanze"],
    "mark": [78, 54, 91, 66],
})
print(df)
```

The data for this week

This week we use 200 sales from an invented Kigali shop. Run this setup cell at the start of every Week 2 lesson:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 200
prices = {"Rice 5kg": 7500, "Cooking oil 1L": 3200, "Sugar 1kg": 1600,
          "Milk 1L": 800, "Airtime 1000": 1000}
products = rng.choice(list(prices), size=n, p=[0.2, 0.15, 0.2, 0.3, 0.15])
sales = pd.DataFrame({
    "date": pd.to_datetime("2026-01-01") + pd.to_timedelta(rng.integers(0, 181, n), unit="D"),
    "district": rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], size=n),
    "product": products,
    "quantity": rng.integers(1, 7, n),
    "payment": rng.choice(["MoMo", "Cash", "Card"], size=n, p=[0.55, 0.35, 0.10]),
})
sales["unit_price"] = sales["product"].map(prices)
sales["total"] = sales["quantity"] * sales["unit_price"]
sales = sales.sort_values("date").reset_index(drop=True)
```

The data is **simulated**: it looks realistic, and everyone gets the same numbers.

Look before you touch

Always inspect new data first:

```
print(sales.head())        # first 5 rows
print(sales.shape)         # (rows, columns)
sales.info()               # column names, types, missing values
print(sales.describe())    # count, mean, min, max of number columns
```

To read a real file you would use `pd.read_csv("file.csv")` or `pd.read_excel("file.xlsx")`. In Colab, upload the file with the folder icon first.

Selecting columns and rows

```
print(sales["product"].head())                  # one column (a Series)
print(sales[["product", "total"]].head())       # several columns
print(sales.loc[0])                              # row with label 0
print(sales.iloc[:3])                            # first 3 rows by position
```

Filtering rows

```
momo = sales[sales["payment"] == "MoMo"]
print(len(momo))

big_rice = sales[(sales["product"] == "Rice 5kg") & (sales["quantity"] >= 5)]
print(big_rice.head())

kigali_east = sales[sales["district"].isin(["Gasabo", "Kicukiro"])]
```

Use `&` for "and" and `|` for "or", with **brackets around each condition**.

Sorting

```
top = sales.sort_values("total", ascending=False).head(5)
print(top[["date", "product", "quantity", "total"]])
```

New columns

```
sales["total_with_vat"] = (sales["total"] * 1.18).round()
sales["month"] = sales["date"].dt.month
sales["big_sale"] = sales["total"] >= 20000
print(sales.head())
```

Each new column is calculated for all 200 rows at once — vectorised, like NumPy.

Quick counts

```
print(sales["payment"].value_counts())
print(sales["product"].value_counts(normalize=True).round(2))   # shares
print(sales["total"].sum())
```

Key terms

- **DataFrame** — a Pandas table with named columns.
- **Series** — one column of a DataFrame.

- **head / info / describe** — the first three things to run on new data.
- **loc / iloc** — select rows by label / by position.
- **Filter** — keep rows that meet a condition, e.g. `df[df["mark"] >= 50]`.
- **value_counts** — count how often each value appears.

Remember this: Load data into a DataFrame, inspect it with head, info and describe, then select, filter, sort and add columns. These few commands cover most everyday data work.

Name: _____ Date: _____

Activity

Task 1

How many sales were paid by Card?

Task 2

Show the 5 smallest sales by `total`.

Task 3

Create a column `price_band` that is `"high"` when `unit_price` ≥ 3000 and `"low"` otherwise. (Hint: `np.where(condition, "high", "low")`.)

Task 4

What share of all sales happened in Nyarugenge?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a Pandas DataFrame?

   A. A type of loop
   B. A table of data with named columns
   C. A picture frame
   D. A chart

2. Which command shows the column types and missing values?

   A. df.sort_values()
   B. df.head()
   C. df.plot()
   D. df.info()

3. How do you keep only rows where payment is MoMo?

   A. sales["MoMo"]
   B. sales.loc["MoMo"]
   C. sales.MoMo()
   D. sales[sales["payment"] == "MoMo"]

4. How do you combine two filter conditions with 'and' in Pandas?

   A. Use & with brackets around each condition
   B. Use the word and
   C. Use a comma
   D. Use +

5. What does `sales["payment"].value_counts()` show?

   A. The first 5 rows
   B. The column type
   C. The total income
   D. How many sales used each payment method

6. What does `sales.sort_values("total", ascending=False).head(5)` give?

   A. 5 random sales
   B. The 5 smallest sales
   C. The 5 largest sales
   D. The first 5 rows unsorted

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.